ICS Exercise — Week 8: Solutions

May 8, 2026

Problem 1: I/O Tracing with `fork` and `dup2`

1.1 Final on-disk file contents

File	Final contents	Length
`/work/notes.txt`	`ABCDE`	5
`/work/log.txt`	`XY1p1c12`	8
`/work/out.txt`	`ABCDZ`	5

`notes.txt` was opened `O_RDONLY` and is never written. The other two files are built up event by event:

Line	Event	After this event
13	parent `Write(fd5, "AB", 2)`, OFT-out offset 0 → 2	`out = "AB"`
14	parent `Write(fd4, "1", 1)`, `O_APPEND` seeks to end (=2), then writes; OFT-log offset 2 → 3	`log = "XY1"`
20	child `Write(fd5, "CD", 2)` at shared OFT-out offset 2; OFT-out 2 → 4	`out = "ABCD"`
23	child `Write(fd5, "Z", 1)` at OFT-out offset 4; OFT-out 4 → 5	`out = "ABCDZ"`
24	child `exit(0)` flushes child stdio buffer `"p1c1"` through fd 1, which `Dup2` rewired to OFT-log; `O_APPEND` seeks to end (=3); OFT-log 3 → 7	`log = "XY1p1c1"`
30	parent `Write(fd4, "2", 1)`, `O_APPEND` seeks to end (=7); OFT-log 7 → 8	`log = "XY1p1c12"`

Two subtle points worth highlighting:

1. The child's `printf("c1")` on line 22 only deposits two bytes into the **child's** stdio user-space buffer. Because of the preceding `Fork()`, that buffer already contained the `"p1"` that the parent had buffered on line 15 — so when the child's buffer is finally flushed at `exit(0)`, the bytes that hit the kernel via `write()` are the four bytes `"p1c1"`, not just `"c1"`. The child's `Dup2` on line 21 has redirected fd 1 to OFT-log, so those four bytes land in `log.txt`.
2. Each `O_APPEND` write atomically seeks to end-of-file before writing. The OFT offset visible through `fd4` ends up equal to the file size after the write, but the seek-to-end is what determines where the bytes land — never the OFT offset that was there on entry.

1.2 Stdout (terminal) output

The terminal sees exactly the 7 bytes:

```
p1p2:E
```

(That is: `p`, `1`, `p`, `2`, `:`, `E`, `\n`.)

Why:

1. The child contributes **nothing** to the terminal. By the time the child's stdio buffer is flushed (at `exit(0)`), the child's fd 1 has been rewired by `Dup2` on line 21 to the OFT entry of `log.txt`. The buffer's contents (`"p1c1"`) flow through fd 1 → OFT-log → `log.txt`, not through any descriptor that points at the terminal.
2. The parent's stdio `stdout` buffer holds `"p1"` from line 15 across the `Fork()`. The parent's fd 1 was *not* touched (only the child redirected its own fd 1), so the parent's stdio still routes through fd 1 → OFT-tty → terminal. On line 31, `printf("p2:%s\n", buf)` appends `"p2:E\n"` to the buffer. Because the parent's stdout is line-buffered (it is connected to a terminal), the trailing `\n` triggers a flush — and the *full* buffer `"p1p2:E\n"` lands on the terminal in one `write()` syscall.
3. `Wait(NULL)` on line 27 ensures the child has terminated before the parent's read on line 28, so there is no possible interleaving where parent output could land between child events.

1.3 Three-table-model diagnostic

(a) Reference count of the OFT entry behind `log.txt` after line 21.

The reference count is **3**. Three descriptor-table slots, across both processes, point at this single OFT entry:

- the parent's `fd 4` (created by `Open` on line 9),
- the child's `fd 4` (inherited from the parent at `Fork()` on line 17),
- the child's `fd 1` (rewired by `Dup2(fd4, STDOUT_FILENO)` on line 21).

`Dup2` does **not** allocate a new OFT entry. It tears down whatever the child's `fd 1` previously pointed at (the OFT-tty entry that the shell handed the child) and aliases the child's `fd 1` slot onto the same OFT entry that the child's `fd 4` already references. That is precisely why the file offset and the `O_APPEND` flag are shared between the child's `fd 4` and the child's `fd 1` — both descriptor-table slots index the same OFT row, which is itself a single row.

(b) Why the parent's read on line 28 returns `'E'`, not `'C'`.

The file offset for an open file lives in the **open file table** entry, not in the per-process descriptor table and not in the system-wide v-node table. The descriptor table maps a small integer to an OFT entry; the v-node table holds metadata (file size, permissions, on-disk locator) that is independent of any open. The OFT layer sits in between specifically to hold the per-`open()`-call cursor.

`Open` on line 8 produced a single OFT-notes entry with offset 0. `Fork()` on line 17 caused the parent's and the child's `fd 3` to *share* OFT-notes (the descriptor table is duplicated, but the entries it points at are not duplicated — they gain a second reference). The parent's read on line 12 advanced OFT-notes.offset 0 → 2; the child's read on line 19 advanced the same OFT-notes.offset 2 → 4. When the parent reads on line 28, it consults OFT-notes.offset (now 4) and reads the byte at index 4 of `notes.txt`, which is `'E'`. If the offset had instead lived per descriptor-table or per v-node, the parent would either still see offset 2 (and read `'C'`) or would observe a different shared-but-not-fork-aware value entirely. The OFT layer is exactly where Unix puts the cursor so that fork-shared and `dup2`-shared descriptors share a position while two independent `open()` calls on the same file get independent positions.

Problem 2: Robust I/O on a TCP echo server

2.1 Bug hunt

Any two of the following four; the first two are the headline answers.

1. **Stream-direction switching without `fflush` / `fseek` / `fsetpos`.** ISO C requires that when a `r+` stream alternates

input and output operations, the program must intercept the transition with a call from the seek/flush family — without it the behavior is undefined. `echo_stdio` calls `fgets` (input) and then `fputs` (output) every iteration with nothing between them. The standard's escape hatch is `fseek` (or `rewind`/`fsetpos`), but **on a TCP socket every seek-class call fails** with `ESPIPE` — the descriptor is non-seekable. So the bug has no portable repair while the function remains stdio-based.

2. **Full output buffering on a non-terminal descriptor.** When stdio sees that the underlying fd is not a terminal (and a TCP socket is not), it switches the output side to **fully buffered**. `fputs(line, stream)` deposits bytes into a user-space buffer that is flushed only when it fills, when `fclose` runs, or when the program calls `fflush` explicitly. An interactive client typing one line at a time will type, see no echo, type again, see no echo… and eventually deadlock when both ends are waiting on the other.
3. **Async-signal-unsafe.** Standard I/O acquires per-`FILE*` locks and manipulates user-space buffers, none of which are guaranteed safe inside a signal handler. If a SIGPIPE handler (or any other handler) wanted to log via this code path, it could deadlock against `main`'s pending stdio lock.
4. **`fclose` semantics on `fdopen`'d descriptors.** `fclose(stream)` will close `connfd` itself. Whether that is a defect depends on caller expectations; if the caller plans to reuse `connfd` after `echo_stdio` returns, it has been silently invalidated.

2.2 Rewrite with RIO

```
void echo_rio(int connfd) {
    rio_t rio;
    char buf[MAXLINE];
    ssize_t n;

    rio_readinitb(&rio, connfd);
    while ((n = rio_readlineb(&rio, buf, MAXLINE)) > 0) {
        rio_writen(connfd, buf, n);
    }
}
```

Why this is correct:

- `rio_readinitb` binds a fresh `rio_t` to `connfd`. Every subsequent `rio_readlineb` consults the buffered state in `rio` (not in libc's stdio internals).
- `rio_readlineb` returns bytes-read on success (the `> 0` branch), 0 at EOF (the client called `close` or `shutdown(SHUT_WR)`), or -1 on error. Treating both 0 and -1 as loop exit via the single `> 0` test handles both correctly.
- `rio_writen` is unbuffered and retries internally on `EINTR` and on short writes, so partial-write handling is correct without any application code.
- There is no user-space output buffer to be left stranded — every echoed line goes to the kernel before the next `rio_readlineb` runs.

2.3 `rio_readlineb` buffer trace

Call	Returned `buf`	Return	`rio_cnt` after	`bufptr` offset after	# refills
1	`HEL\n`	4	2	4	1
2	`WORLD\n`	6	0	4	1
3	`FOO\n`	4	5	4	1
4	`BAR\n`	4	1	8	0
5	`BAZ\n`	4	0	1	2
6	(empty)	0	0	1	1 (EOF)

Note: each `rio_read` refill resets `rio_bufptr` back to `&rio_buf[0]`, so the "`bufptr` offset after" column is **not** monotonic — it jumps back to 0 every time a refill happens, then advances as bytes are drained.

Step-by-step

Call 1. `rio_cnt = 0` on entry, so `rio_read` issues refill #1. `read()` returns 6 bytes `"HEL\nWO"`; `rio_buf[0..6) = "HEL\nWO"`, `rio_bufptr = &rio_buf[0]`, `rio_cnt = 6`. The line-reading loop drains one byte at a time: `H` (cnt 5, off 1), `E` (cnt 4, off 2), `L` (cnt 3, off 3), `\n` (cnt 2, off 4) → break on newline. Return 4. After: `rio_cnt = 2`, offset = 4. The two unread bytes `WO` remain in `rio_buf[4..6)`.

Call 2. Drains the leftover `WO` first: `W` (cnt 1, off 5), `O` (cnt 0, off 6). Now `rio_cnt = 0`, so `rio_read` issues refill #2. `read()` returns 4 bytes `"RLD\n"`; `rio_buf[0..4) = "RLD\n"`, `rio_bufptr = &rio_buf[0]`, `rio_cnt = 4`. Drain: `R` (cnt 3, off 1), `L` (cnt 2, off 2), `D` (cnt 1, off 3), `\n` (cnt 0, off 4) → break. Return 6 (`WORLD\n`). After: `rio_cnt = 0`, offset = 4.

Call 3. `rio_cnt = 0`, so refill #3. `read()` returns 9 bytes `"FOO\nBAR\nB"`; `rio_buf[0..9)`, offset reset to 0, `rio_cnt = 9`. Drain: `F` (cnt 8, off 1), `O` (cnt 7, off 2), `O` (cnt 6, off 3), `\n` (cnt 5, off 4) → break. Return 4. After: `rio_cnt = 5`, offset = 4. The five unread bytes `BAR\nB` remain in `rio_buf[4..9)`.

Call 4. No refill — five bytes are already buffered. Drain: `B` (cnt 4, off 5), `A` (cnt 3, off 6), `R` (cnt 2, off 7), `\n` (cnt 1, off 8) → break. Return 4. After: `rio_cnt = 1`, offset = 8. One unread byte `B` remains in `rio_buf[8]`.

Call 5. Drains the leftover `B`: cnt 0, off 9. `rio_cnt = 0`, so refill #4. `read()` returns 2 bytes `"AZ"`; `rio_buf[0..2)`, offset reset to 0, `rio_cnt = 2`. Drain: `A` (cnt 1, off 1), `Z` (cnt 0, off 2). `rio_cnt = 0` again, so refill #5. `read()` returns 1 byte `"\n"`; `rio_buf[0..1)`, offset reset to 0, `rio_cnt = 1`. Drain: `\n` (cnt 0, off 1) → break. Return 4 (`BAZ\n`). After: `rio_cnt = 0`, offset = 1. **This call performed two refills inside a single `rio_readlineb` invocation** — the most subtle row in the table.

Call 6. `rio_cnt = 0`, so `rio_read` issues refill #6. `read()` returns 0 (EOF). The `rio_read` helper returns 0 immediately. Inside `rio_readlineb`, `n` is still 1 (no bytes copied yet), so the function returns 0 without touching `buf` (other than the null terminator). After: `rio_cnt = 0`, offset = 1 (unchanged from Call 5; nothing was drained or refilled successfully).

Sanity check

- Total `read()` syscalls: 6 (one per refill, one of which returned EOF). Same as the count given in the problem statement.
- Total payload bytes returned across calls 1–5: 4 + 6 + 4 + 4 + 4 = 22, which matches the 22-byte input stream.
- The number of refills *per call* is 1 / 1 / 1 / 0 / 2 / 1: **one `rio_readlineb` call is not the same as one `read()` syscall**, in either direction. Call 4 makes zero kernel calls; Call 5 makes two.